

Кафедра
статистики и
кибернетики
*Department of statistics and
cybernetics*

Лекция №3

«Быстрая сортировка. Сортировка слиянием. Очереди»

Лектор: Вадим Владимирович Демичев
доцент, кандидат экономических наук

Москва, 2026

Разделяй и властвуй

Разделяй и властвуй (англ. divide and conquer) в информатике — парадигма разработки алгоритмов, заключающаяся в рекурсивном разбиении решаемой задачи на две или более подзадачи того же типа, но меньшего размера, и комбинировании их решений для получения ответа к исходной задаче; разбиения выполняются до тех пор, пока все подзадачи не окажутся элементарными.

Быстрая сортировка

Быстрая сортировка относится к алгоритмам сортировки.

Она работает намного быстрее сортировки выбором и часто применяется в реальных программах.

Например, в стандартную библиотеку C входит функция с именем `qsort`, реализующая быструю сортировку.

Быстрая сортировка также основана на стратегии разделяй и властвуй.

Массивы, которые не нужно сортировать

СОРТИРОВАТЬ
ТАКИЕ МАССИВЫ
НЕ НУЖНО

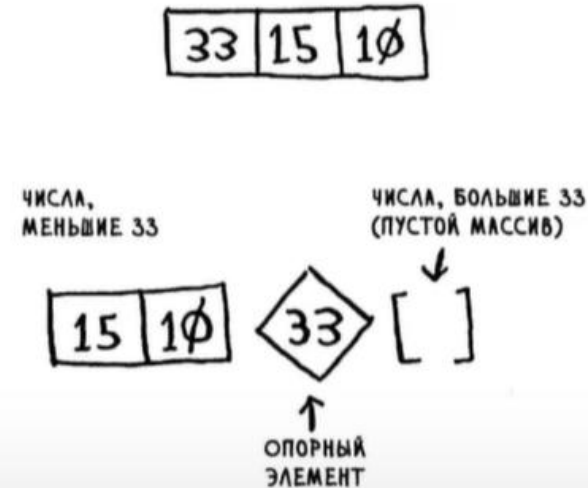
{ [] ← ПУСТОЙ МАССИВ
[20] ← МАССИВ С ОДНИМ ЭЛЕМЕНТОМ

```
def quick sort(array):  
    if len(array) < 2:  
        return array
```

Быстрая сортировка

Этот процесс называется разделением. Имеем:

- подмассив всех элементов, меньших опорного;
- о опорный элемент;
- о подмассив всех элементов, больших опорного.



Быстрая сортировка

Если бы подмассивы были отсортированы, то их можно было бы объединить в порядке левый подмассив - опорный элемент - правый подмассив и получить отсортированный массив .



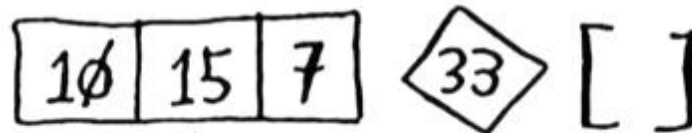
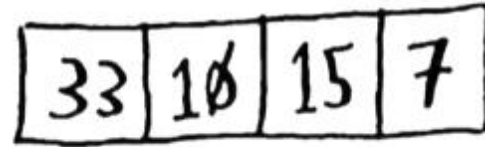
```
quicksort([15, 10]) + [33] + quicksort([])  
> [10, 15, 33]    ←..... Отсортированный массив
```

Еще пример

- Этот метод работает при любом опорном элементе. Допустим, вместо 33 в качестве опорного был выбран элемент 15.
Алгоритм:
- 1. Выбрать опорный элемент.
- 2. Разделить массив на два подмассива: элементы, меньшие опорного, и элементы, большие опорного.
- 3. Рекурсивно применить быструю сортировку к двум подмассивам.



Еще один пример:



Другой пример

3 5 2 1 4

[] 1 3 2 5 4

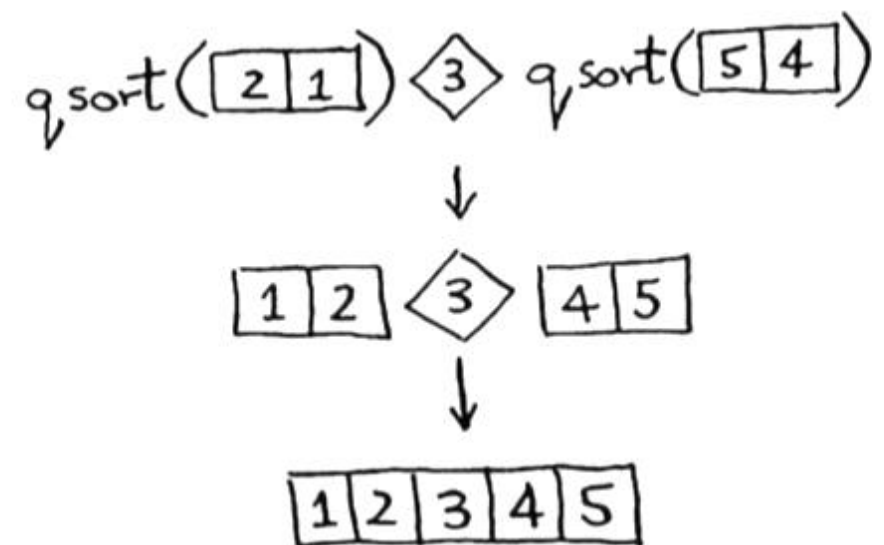
1 2 3 5 4

2 1 3 5 4

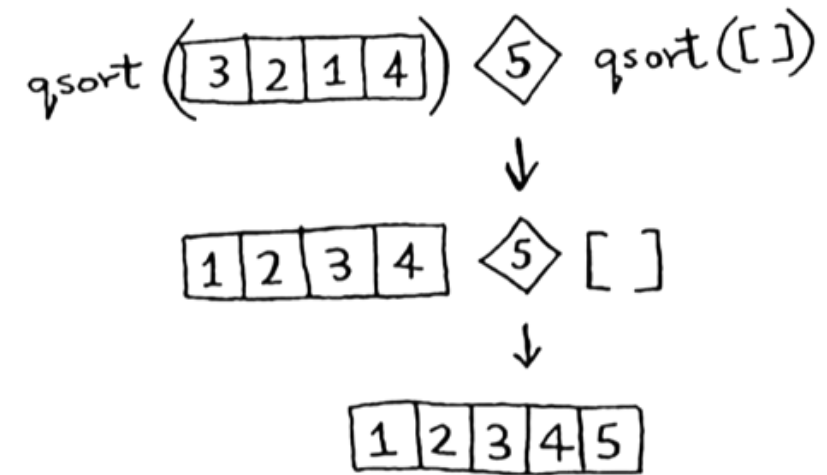
3 2 1 4 5

3 2 1 4 5 []

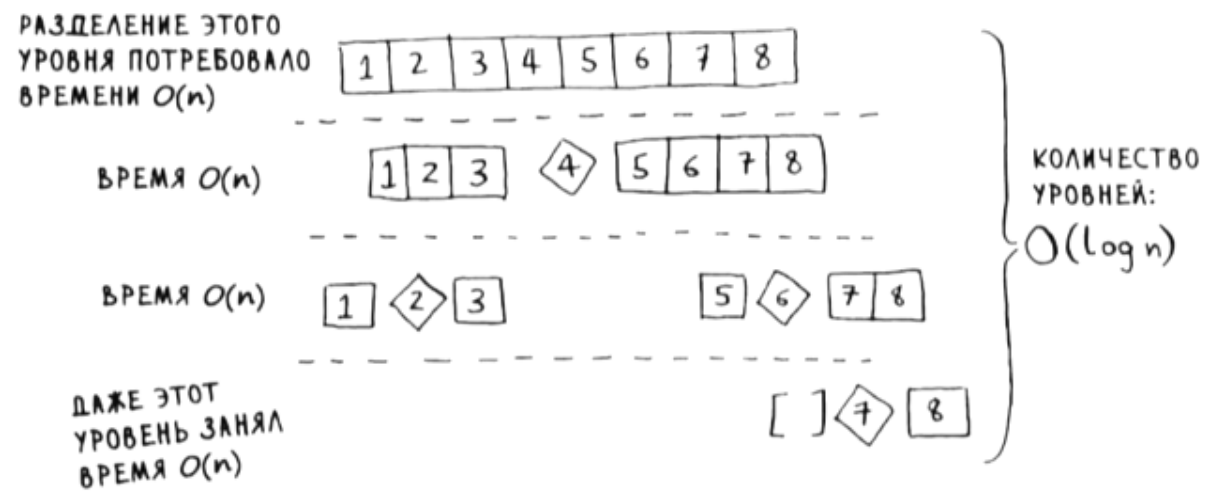
Другой пример



Другой пример



Сложность



Пример - продолжение:

```
qsort.py* x
1  def quicksort(array):
2      if len(array) < 2:
3          return array #<<базовый случай:
4                          #массивы с 0 и 1 элементом уже "отсортированы"
5      else:
6          pivot = array[0]#<<Рекурсивный случай
7          less = [i for i in array[1:] if i<=pivot] #подмассив элементов меньше опорного
8          greater = [i for i in array[1:] if i > pivot]#подмассив всех
9                                     #элементов, больше опорного
10         return quicksort(less) + [pivot] + quicksort(greater)
11  m = [10,5,2,3]
12
13  print(quicksort(m))
14
```

Сортировка слиянием

- При сортировке слиянием мы разделяем массив пополам до тех пор, пока каждый участок не станет длиной в один элемент.
- Затем эти участки возвращаются на место (сливаются) в правильном порядке.

3	8	2	1	5	4	6	7
---	---	---	---	---	---	---	---

3	8	2	1
---	---	---	---

5	4	6	7
---	---	---	---

3	8	2	1	5	4	6	7
3	8	2	1	5	4	6	7

Слияние

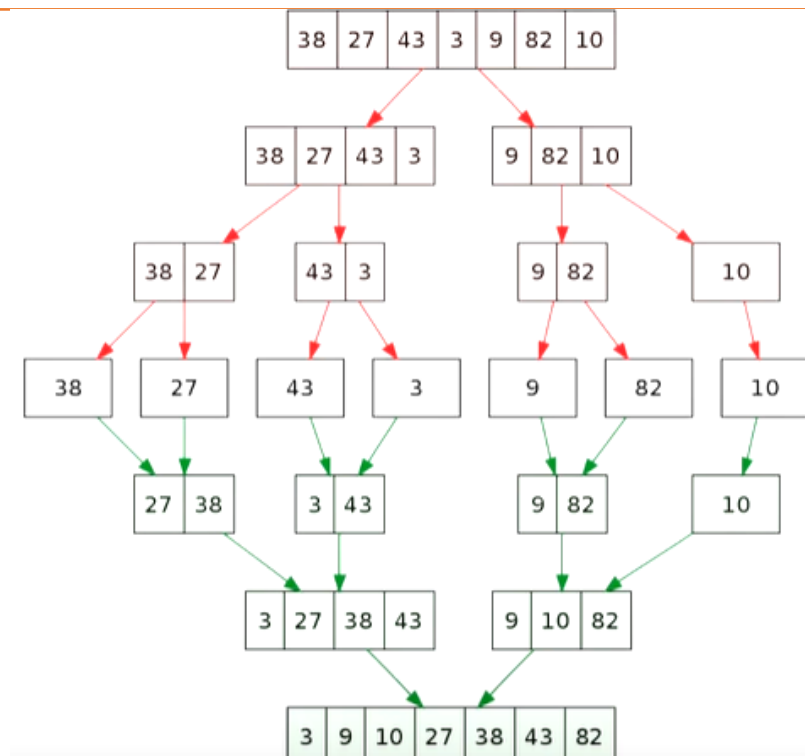
3 8 2 1 5 4 6 7

3 8 1 2 4 5 6 7

1 2 3 8 4 5 6 7

1 2 3 4 5 6 7 8

Слияние



Реализация (слияние)

```
merge_sort.py* x
1  def merge(left, right):
2      result = []
3      i, j = 0, 0
4      while i < len(left) and j < len(right):
5          if left[i] < right[j]:
6              result.append(left[i])
7              i += 1
8          else:
9              result.append(right[j])
10             j += 1
11     if left:
12         result = result + left[i:]
13     if right:
14         result = result + right[j:]
15
16     return result
```

Реализация (слияние)

```
merge_sort.py* x
1  def merge(left, right):
2      result = []
3      i, j = 0, 0
4      while i < len(left) and j < len(right):
5          if left[i] < right[j]:
6              result.append(left[i])
7              i += 1
8          else:
9              result.append(right[j])
10             j += 1
11     if left:
12         result = result + left[i:]
13     if right:
14         result = result + right[j:]
15
16     return result
```

```
18
19     def merge_sort(L):
20         if len(L) < 2:
21             return L[:]
22         else:
23             middle = int(len(L)/2)
24             left = merge_sort(L[:middle])
25             right = merge_sort(L[middle:])
26             return merge(left, right)
27
28
29     mas = [5, 3, 4, 7, 10]
30
31     print(merge_sort(mas))
```

Сравнение сложности алгоритмов сортировки

Алгоритм	Структура данных	Временная сложность		
		Лучшее	В среднем	В худшем
Быстрая сортировка	Массив	$O(n \log(n))$	$O(n \log(n))$	$O(n^2)$
Сортировка слиянием	Массив	$O(n \log(n))$	$O(n \log(n))$	$O(n \log(n))$
Пирамидальная сортировка	Массив	$O(n \log(n))$	$O(n \log(n))$	$O(n \log(n))$
Пузырьковая сортировка	Массив	$O(n)$	$O(n^2)$	$O(n^2)$
Сортировка вставками	Массив	$O(n)$	$O(n^2)$	$O(n^2)$
Сортировка выбором	Массив	$O(n^2)$	$O(n^2)$	$O(n^2)$
Блочная сортировка	Массив	$O(n+k)$	$O(n+k)$	$O(n^2)$
Поразрядная сортировка	Массив	$O(nk)$	$O(nk)$	$O(nk)$

Сравнение сортировок

[Sorting Algorithms Animations](#)

ОЧЕРЕДИ

- Очередь работает точно так же, как и в реальной жизни.
- Предположим, вы с другом стоите в очереди на автобусной остановке. Если вы стоите ближе к началу очереди, то вы первым сядете в автобус.
- Структура данных очереди работает аналогично.
- Очереди чем-то похожи на стеки: вы не можете обращаться к произвольным элементам очереди.
- Вместо этого поддерживаются всего две операции: постановка в очередь и извлечение из очереди.



Принцип First In First Out (FIFO)

Очередь относится к категории структур данных FIFO: First In, First Out («первым вошел, первым вышел»). А стек принадлежит к числу структур данных LIFO: Last In, First Out («последним пришел, первым вышел»).



Преимущества использования очередей

Слабое связывание — очереди сообщений создают неявные интерфейсы обмена данными, которые позволяют процессам быть независимыми друг от друга т.е вы просто определяете формат сообщений отправляемых от одного процесса другому.

Масштабируемость — очереди сообщений позволяют распределить процессы обработки информации. Таким образом, они позволяют легко наращивать скорость, с которой сообщения добавляются в очередь и обрабатываются.

Эластичность и возможность выдерживать пиковые нагрузки — очереди сообщений могут выполнять роль своего рода буфера для накопления данных в случае пиковой нагрузки, смягчая тем самым нагрузку на систему обработки информации и не допуская ее отказа.

Преимущества использования очередей:

- **Отказоустойчивость** — очереди сообщений позволяют отделить процессы друг от друга, так что если процесс, который обрабатывает сообщения из очереди падает, то сообщения могут быть добавлены в очередь на обработку позднее, когда система восстановится.
- **Гарантированная доставка** — использование очереди сообщений гарантирует, что сообщение будет доставлено и обработано в любом случае (пока есть хотя бы один обработчик).

Преимущества использования очередей

- **Гарантированный порядок доставки** — большая часть систем очередей сообщений способны обеспечить гарантии того, что данные будут обрабатываться в определённом порядке (чаще всего в том порядке в котором они поступили).
- **Буферизация** — очереди сообщений позволяет отправлять и получать сообщения при этом работая с максимальной эффективностью, предлагая буферный слой — процесс записи в очередь может происходить настолько быстро, насколько быстро это в состоянии выполнить очередь сообщений, а не обработчик сообщения.

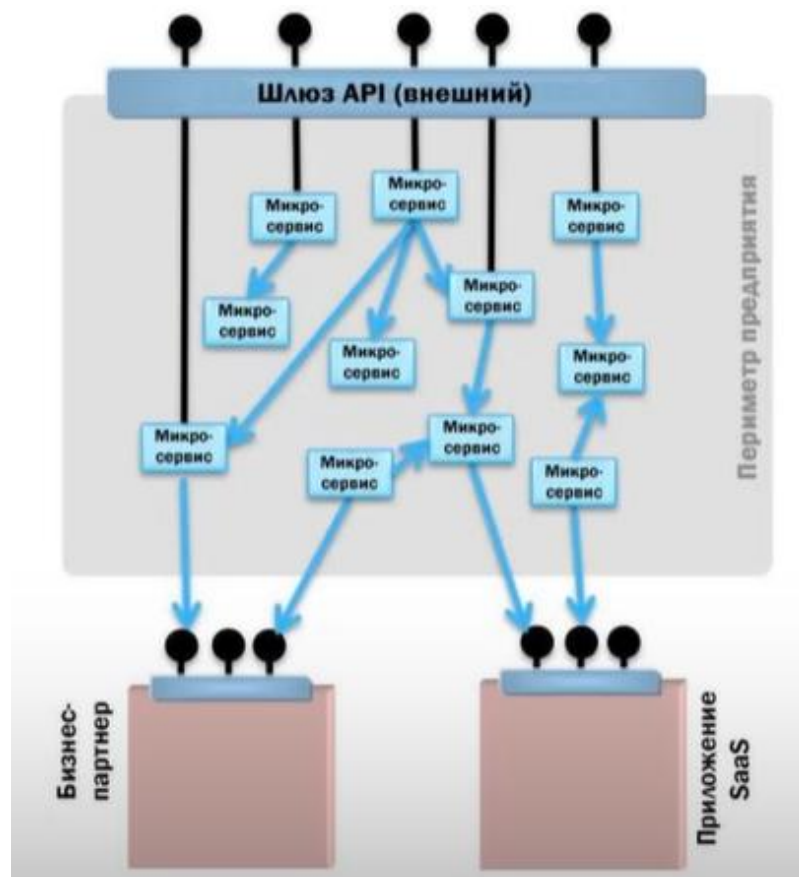
Преимущества использования очередей

- **Понимание потоков данных** — очереди сообщений позволяют выявлять узкие места в потоках данных приложения, легко можно определить какая из очередей забивается, какая простаивает и определить что необходимо делать — добавлять новых обработчиков сообщений или оптимизировать текущую архитектуру.
- **Асинхронная связь** — очереди сообщений предоставляют возможность асинхронной обработки данных, которая позволяет поместить сообщение в очередь без обработки, позволяя системе обработать сообщение позднее, когда появится возможность.

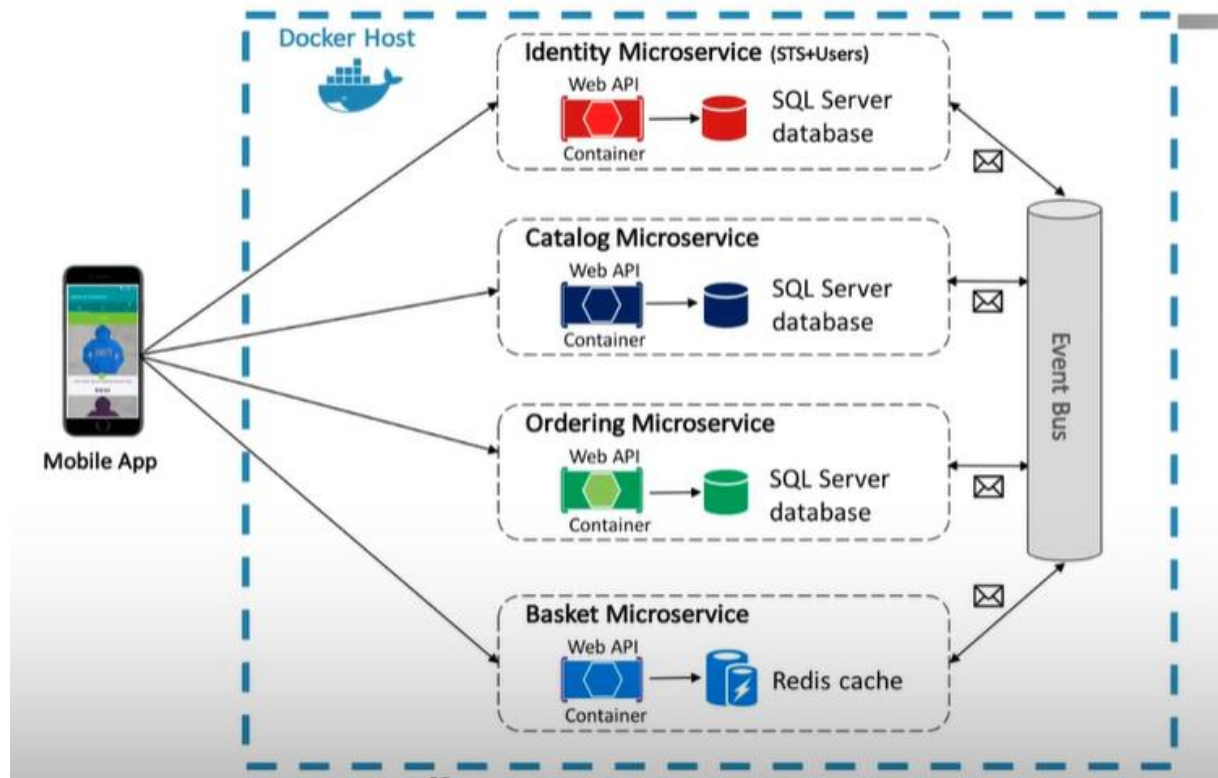
Области применения

- Обработку данных
- Буферизацию потоков данных
- Управление процессами
- Интеграцию и взаимодействие систем

Микросервисная архитектура



Микросервисная архитектура

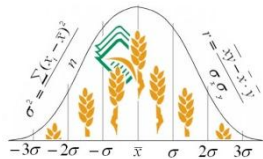


Списки и Массивы

Как работает память

- Представьте, что вы пришли в театр и хотите оставить свои личные вещи в гардеробе. Для хранения вещей есть специальные ящики.
- В каждом ящике помещается один предмет. Вы хотите сдать на хранение две вещи, поэтому требуете выделить вам два ящика.
- В сущности, именно так работает память вашего компьютера. Она представляет собой нечто вроде огромного шкафа с множеством ящиков, и у каждого ящика есть адрес.





Тезисы

- Каждый раз, когда вы хотите сохранить в памяти отдельное значение, вы запрашиваете у компьютера место в памяти, а он выдает адрес для сохранения значения.
- Если же вам понадобится сохранить несколько элементов, это можно сделать двумя основными способами: воспользоваться **массивом** или **списком**.
- **Не существует единственно верного способа сохранения данных** на все случаи жизни, поэтому вы должны знать, чем различаются разные способы.

Массивы

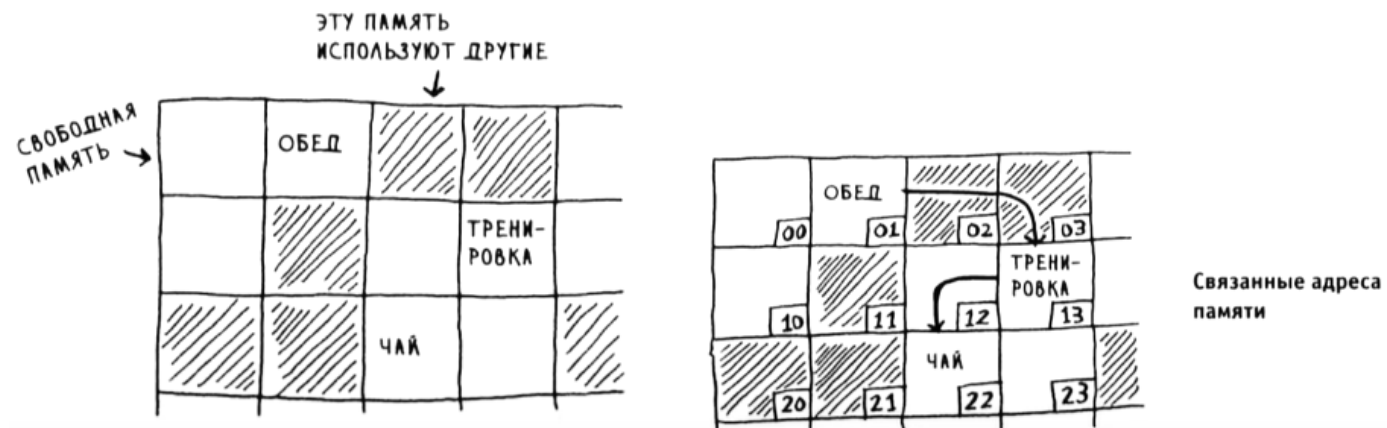
- Представьте, что вы пошли в кино с друзьями и нашли места для своей компании, но тут приходит еще один друг, и ему сесть уже некуда. Приходится искать новое место, где смогут разместиться все. В этом случае вам придется запросить у компьютера другой блок памяти, в котором поместятся все четыре задачи, а потом переместить все свои задачи туда.
- Если вдруг придет еще один друг, места опять не хватит, и вам всем придется перемещаться снова! Сплошная суeta. Кроме того, добавление новых элементов в массив станет серьезной проблемой. Если свободного места нет и вам каждый раз приходится перемещаться в новую область в памяти, операция добавления нового элемента будет выполняться очень медленно. Простейшее решение - «бронирование мест»: даже если список состоит задач всего из 3 задач, вы запрашиваете у компьютера место на 10 позиций... просто на всякий случай. Тогда в список можно будет добавить до 10 задач, и ничего перемещать не придется.

Недостатки

- Лишнее место может не понадобиться, и тогда память будет расходоваться неэффективно. Вы ее не используете, однако никто другой ее использовать тоже не может.
- Если в список будет добавлено более 10 задач, перемещаться все равно придется.

Связанные списки

- При использовании связанного списка элементы могут размещаться где угодно в памяти.
- В каждом элементе хранится адрес следующего элемента списка. Таким образом, набор произвольных адресов памяти объединяется в цепочку.



Односвязный линейный список

- Каждый узел ОЛС содержит 1 поле указателя на следующий узел. Поле указателя последнего узла содержит нулевое значение (указывает на NULL).



Тезисы

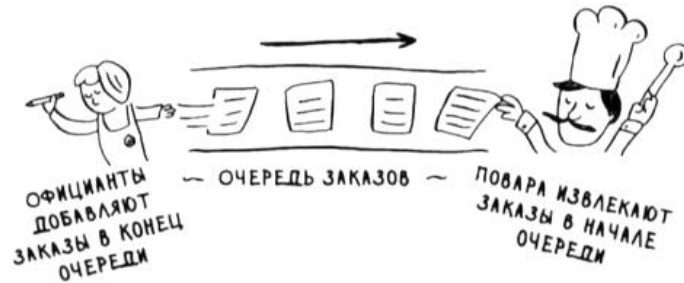
- Связанные списки отлично подходят в тех ситуациях, когда данные должны читаться последовательно: сначала вы читаете один элемент, по адресу переходите к следующему элементу и т. д.
- Но если вы намерены прыгать по списку туда-сюда, держитесь подальше от связанных списков.

Сравнение массивов и списков

<i>Массив</i>	<i>Список</i>
Выделение памяти осуществляется единовременно под весь массив до начала его использования	Выделение памяти осуществляется по мере ввода новых элементов
При удалении/добавлении элемента требуется копирование всех последующих элементов для осуществления их сдвига	Удаление/добавление элемента осуществляется переустановкой указателей, при этом сами данные не копируются
Для хранения элемента требуется объем памяти, необходимый только для хранения данных этого элемента	Для хранения элемента требуется объем памяти, достаточный для хранения данных этого элемента и указателей (1 или 2) на другие элементы списка
Доступ к элементам может осуществляться в произвольном порядке	Возможен только последовательный доступ к элементам

Упражнение 1

- Допустим, вы пишете приложение для приема заказов от посетителей ресторана. Приложение должно хранить список заказов. Официанты добавляют заказы в список, а повара читают заказы из списка и выполняют их. Заказы образуют очередь: официанты добавляют заказы в конец очереди, а повар берет первый заказ из очереди и начинает готовить.
- Какую структуру данных вы бы использовали для реализации этой очереди: массив или связанный список?

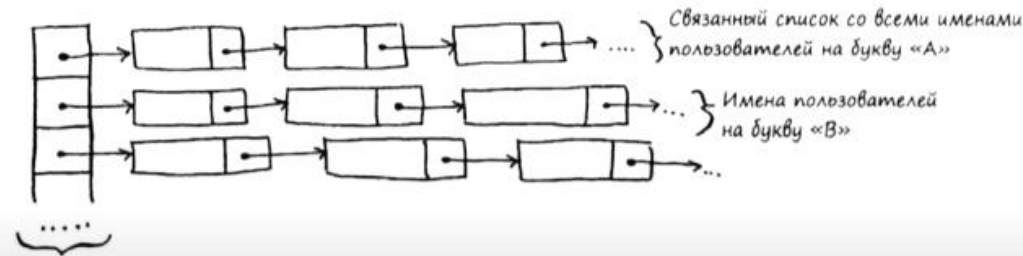


Упражнение 2

- Проведем мысленный эксперимент. Допустим, Вконтакте хранит список имен пользователей. Когда кто-то пытается зайти на сайт Вконтакте, система пытается найти имя пользователя. Если имя входит в список имен зарегистрированных пользователей, то вход разрешается. Пользователи приходят на Вконтакте достаточно часто, поэтому поиск по списку имен пользователей будет выполняться часто. Будем считать, что Вконтакте использует бинарный поиск для поиска в списке. Бинарному поиску необходим произвольный доступ - алгоритм должен мгновенно обратиться к среднему элементу текущей части списка. Зная это обстоятельство, как бы вы реализовали список пользователей: в виде массива или в виде связанного списка?

Упражнение 3

- Рассмотрим гибридную структуру данных: массив связанных списков. Имеется массив из 26 элементов. Каждый элемент содержит ссылку на связанный список. Например, первый элемент массива указывает на связанный список всех имен пользователей, начинающихся на букву «А». Второй элемент указывает на связанный список всех имен пользователей, начинающихся на букву «В», и т. д.
- Предположим, пользователь с именем Adit В регистрируется на Вконтакте и вы хотите добавить его в список. Вы обращаетесь к эле 1 массива, находите связанный список элемента 1 и добавляете Adit В в конец списка. Теперь предположим, что зарегистрировать нужно пользователя Zakhir Н. Вы обращаетесь к элементу 26, который содержит связанный список всех имен, начинающихся с Z и проверяете, присутствует ли Zakhir Н в этом списке. Теперь сравните эту гибридную структуру данных с массивами и связанными списками. Будет ли она быстрее или медленнее каждой исходной структуры при поиске и вставке?



Связанные списки

- **Связный список** является простейшим типом данных динамической структуры, состоящей из элементов (узлов). Каждый узел включает в себя в классическом варианте два поля:
- данные (в качестве данных может выступать переменная, объект класса или структуры и т. д.)
- указатель на следующий узел в списке.



Классификация списков

По количеству полей указателей различают однонаправленный (односвязный) и двунаправленный (двусвязный) списки.

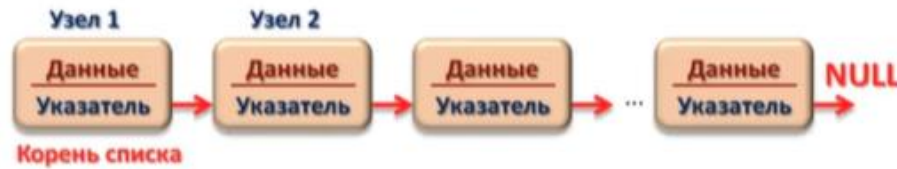
- Связный список, содержащий только один указатель на следующий элемент, называется **односвязным**.
- Связный список, содержащий два поля указателя – на следующий элемент и на предыдущий, называется **двусвязным**.

По способу связи элементов различают линейные и циклические списки.

- Связный список, в котором, последний элемент указывает на NULL, называется **линейным**.
- Связный список, в котором последний элемент связан с первым, называется **циклическим**.

Односвязный линейный список

- Каждый узел ОЛС содержит 1 поле указателя на следующий узел. Поле указателя последнего узла содержит нулевое значение (указывает на NULL).



Односвязный циклический список

- Каждый узел ОЦС содержит 1 поле указателя на следующий узел. Поле указателя последнего узла содержит адрес первого узла (корня списка).



Двусвязный линейный список

- Каждый узел ДЛС содержит два поля указателей: на следующий и на предыдущий узел. Поле указателя на следующий узел последнего узла содержит нулевое значение (указывает на NULL). Поле указателя на предыдущий узел первого узла (корня списка) также содержит нулевое значение (указывает на NULL).



Двусвязный циклический список

- Каждый узел ДЦС содержит два поля указателей: на следующий и на предыдущий узел. Поле указателя на следующий узел последнего узла содержит адрес первого узла (корня списка). Поле указателя на предыдущий узел первого узла (корня списка) содержит адрес последнего.



Инициализация односвязного линейного списка

Инициализация списка предназначена для создания корневого узла списка, у которого поле указателя на следующий элемент содержит нулевое значение.



Добавление узла в односвязном линейном списке

Добавление узла в ОЛС включает в себя следующие этапы:

- создание добавляемого узла и заполнение его поля данных;
- переустановка указателя узла, предшествующего добавляемому, на добавляемый узел;
- установка указателя добавляемого узла на следующий узел (тот, на который указывал предшествующий узел).



Удаление узла в односвязном линейном списке

Удаление узла ОЛС включает в себя следующие этапы:

- установка указателя предыдущего узла на узел, следующий за удаляемым;
- освобождение памяти удаляемого узла.



Инициализация циклического линейного списка

- Инициализация списка предназначена для создания корневого узла списка, у которого поле указателя на следующий элемент содержит адрес самого корневого элемента.



Инициализация двусвязного линейного списка

- Инициализация списка предназначена для создания корневого узла списка, у которого поле указателя на следующий элемент содержит адрес самого корневого элемента.



Добавление узла в двусвязном линейном списке

Добавление узла в ДЛС включает в себя следующие этапы:

- создание узла добавляемого элемента и заполнение его поля данных;
- переустановка указателя "следующий" узла, предшествующего добавляемому, на добавляемый узел;
- переустановка указателя "предыдущий" узла, следующего за добавляемым, на добавляемый узел;
- установка указателя "следующий" добавляемого узла на следующий узел (тот, на который указывал предшествующий узел);
- установка указателя "предыдущий" добавляемого узла на узел, предшествующий добавляемому (узел, переданный в функцию).



Удаление узла в двусвязном линейном списке

Удаление узла ДЛС включает в себя следующие этапы:

- установка указателя "следующий" предыдущего узла на узел, следующий за удаляемым;
- установка указателя "предыдущий" следующего узла на узел, предшествующий удаляемому.



Удаление узла в двусвязном линейном списке

- При замене соседних узлов переустановка указателей выглядит следующим образом:



- При замене узлов, не являющихся соседними переустановка указателей выглядит следующим образом:



Инициализация двусвязного циклического списка

- Инициализация списка предназначена для создания корневого узла списка, у которого поля указателей на следующий и предыдущий узлы указывают на сам создаваемый узел.



Добавление узла в двусвязном циклический список

Добавление узла в ДЦС включает в себя следующие этапы:

- создание узла добавляемого элемента и заполнение его поля данных;
- переустановка указателя "следующий" узла, предшествующего добавляемому, на добавляемый узел;
- переустановка указателя "предыдущий" узла, следующего за добавляемым, на добавляемый узел;
- установка указателя "следующий" добавляемого узла на следующий узел (тот, на который указывал предшествующий узел);
- установка указателя "предыдущий" добавляемого узла на узел, предшествующий добавляемому (узел, переданный в функцию).



Вопросы

